西安电子科技大学网络与信息安全学院
信息与内容安全 实验报告

班　　级：2318039

学　　号：23009201082

姓　　名：魏朝旭

电子邮箱：3100528481@qq.com

指导教师：彭春蕾

2026 年 6 月 20 日

实验题目： PyTorch 基础

实验摘要：

本实验旨在学习 PyTorch 深度学习框架的基础知识与实践操作。实验内容涵盖 Tensor 张量的创建与运算、DataLoader 数据加载机制、Autograd 自动微分原理、神经网络模块的构建方法，以及使用 FashionMNIST 数据集完成一个完整的图像分类任务。通过本实验，掌握 PyTorch 的核心概念和基本使用流程，为后续深度学习模型的开发奠定基础。实验使用 GPU 加速训练（如果可用），并可视化训练过程与预测结果。

题目描述：

PyTorch 是一个开源的深度学习框架，以其动态计算图、易用的 API 和强大的 GPU 加速能力而广受欢迎。本实验基于 PyTorch 官方教程，系统学习以下核心内容：

1. Tensors 基础：理解 PyTorch 中张量(Tensor)的数据结构，掌握创建、索引、切片、变形、运算等操作。

2. DataLoaders：学习使用 torch.utils.data.DataLoader 加载数据集，理解批处理(batch)、打乱(shuffle)等机制。

3. Autograd 自动微分：理解 PyTorch 的自动求导机制，掌握 backward()计算梯度和梯度跟踪控制。

4. 神经网络基础：使用 nn.Module 构建神经网络，理解前向传播、损失函数和优化器。

5. 训练分类器：在 FashionMNIST 数据集上训练一个多层感知机(MLP)

分类器，评估模型性能。

实验参考来源：

https://docs.pytorch.org/tutorials/beginner/basics/intro.html

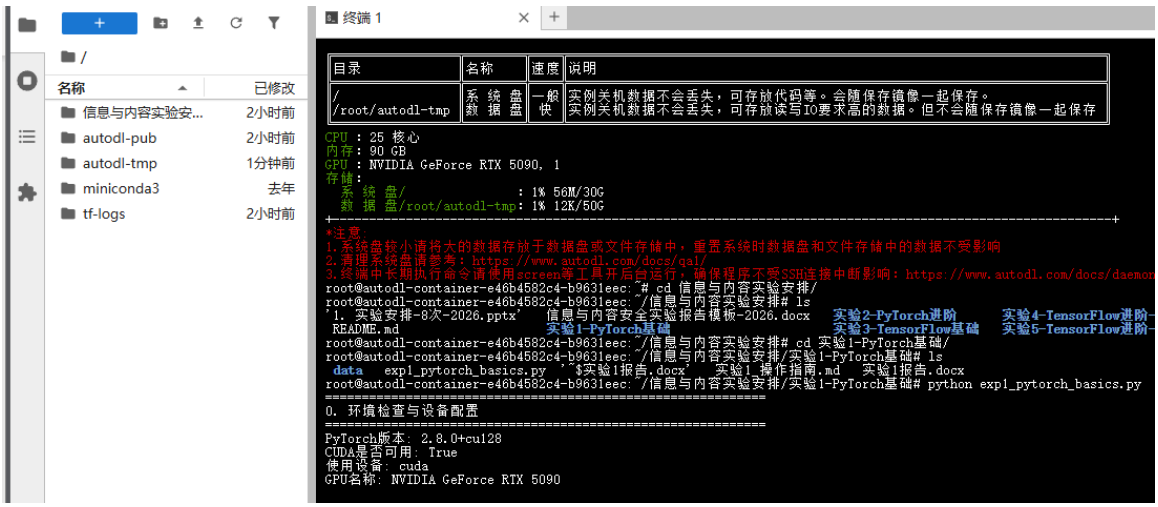
https://docs.pytorch.org/tutorials/domains.html

实验内容：

2.1 环境配置与设备检查

首先检查 **PyTorch** 版本、**CUDA** 可用性及 **GPU** 信息。代码自动检测是否有可用的 **CUDA** 设备，如果有则使用 **GPU** 进行加速，否则回退到 **CPU** 运行。

预期输出：**PyTorch** 版本号、**CUDA** 是否可用、设备类型**(cuda/cpu)**、**GPU** 名称。



[截图 1：环境检查与 Tensor 基础操作输出]

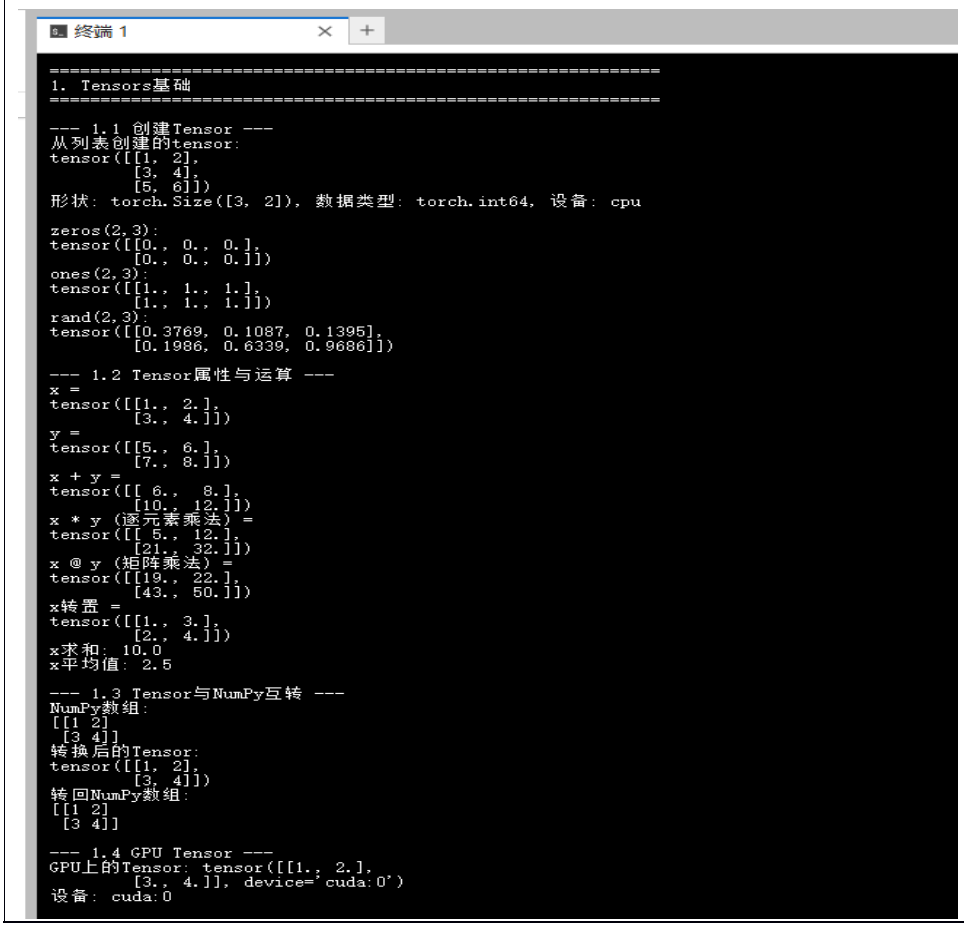
2.2 Tensors 基础操作

Tensor 是 **PyTorch** 中最基本的数据结构，类似于 **NumPy** 的 **ndarray**，但可以在 **GPU** 上运行。本部分学习 **Tensor** 的创建方法（从列表、**zeros**、**ones**、**rand** 等）、属性（**shape**、**dtype**、**device**）和基本运算（加减乘除、矩阵乘法、转置、求和、均值等）。

同时学习 **Tensor** 与 **NumPy** 数组之间的相互转换，以及如何将 **Tensor** 移动到 **GPU** 上进行加速计算。

关键操作包括：

- **torch.tensor()**、**torch.zeros()**、**torch.ones()**、**torch.rand()**
- 逐元素运算：**+**、*****；矩阵乘法：**@**
- **.T** 转置、**.sum()**求和、**.mean()**均值
- **.to(device)**设备转移



[截图 2: Tensor 基础操作输出]

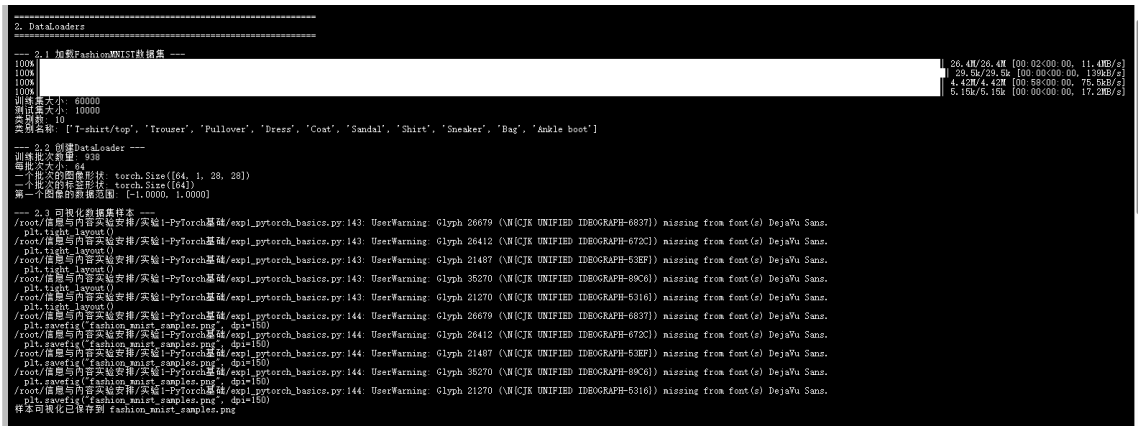
2.3 DataLoader 与数据集加载

使用 **torchvision.datasets** 加载 **FashionMNIST** 数据集，该数据集包含 **60000** 张训练图像和 **10000** 张测试图像，共 **10** 个类别（T 恤、裤子、套头衫、连衣裙、外套、凉鞋、衬衫、运动鞋、包、短靴）。

通过 **transforms.Compose** 组合预处理操作：**ToTensor** 将图像转换为张量，**Normalize** 进行标准化。使用 **DataLoader** 创建可迭代的数据加载器，设置 **batch_size** 和 **shuffle** 参数。

数据集信息：

- 训练集大小：**60000**
- 测试集大小：**10000**
- 图像尺寸：**28×28** 灰度图像
- 类别数：**10**



[截图 3: DataLoader 加载信息与样本可视化图]

2.4 Autograd 自动微分

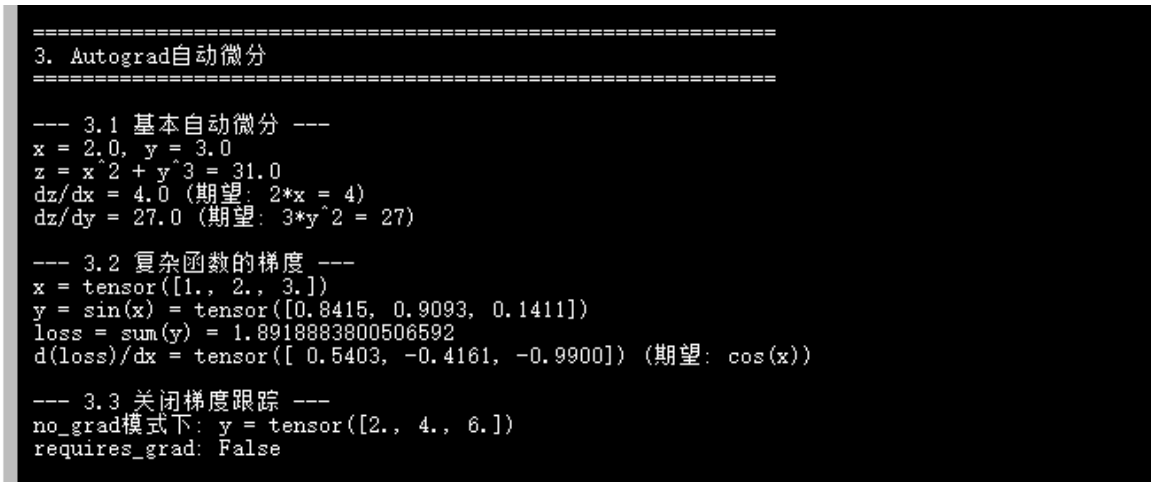
Autograd 是 **PyTorch** 的自动微分引擎，它通过记录张量上的所有操作

构建计算图，然后使用链式法则自动计算梯度。本部分通过具体例子理解自动微分的工作原理。

示例 **1**：计算 **z = x^2 + y^3** 对 **x** 和 **y** 的偏导数。当 **x=2, y=3** 时，**dz/dx=4, dz/dy=27**。

示例 **2**：计算向量函数 **y=sin(x)** 的梯度，验证 **d(sin(x))/dx = cos(x)**。

同时学习使用 **torch.no_grad()**上下文管理器关闭梯度跟踪，这在模型推理时非常有用，可以节省内存并加速计算。



[截图 4：Autograd 自动微分计算结果]

2.5 神经网络基础

使用 **nn.Module** 定义神经网络模型。本实验构建一个多层感知机**(MLP)**，结构如下：

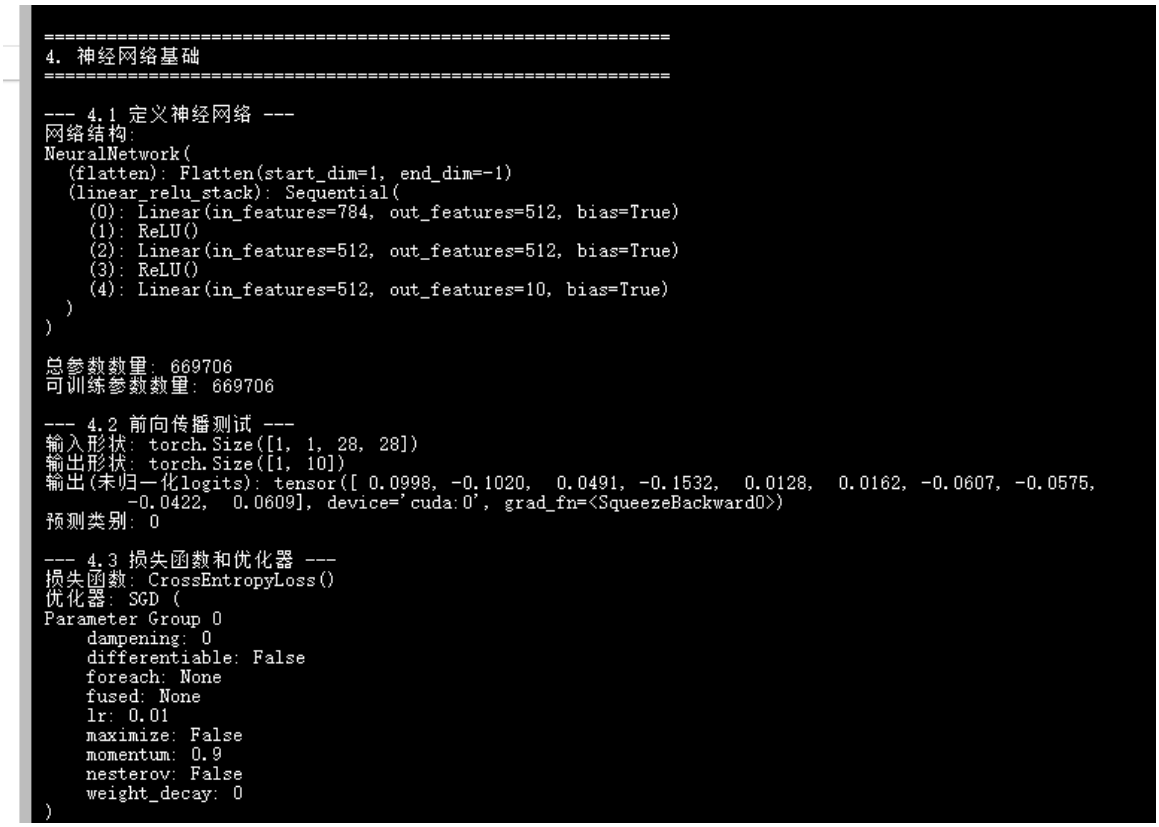
输入层：**28x28=784** 个神经元（**Flatten** 展平）

隐藏层 **1**：**512** 个神经元，**ReLU** 激活

隐藏层 **2**：**512** 个神经元，**ReLU** 激活

输出层：**10** 个神经元（对应 **10** 个类别）

模型总参数数量约为 **669,706** 个。使用 **CrossEntropyLoss** 作为损失函数，**SGD**（随机梯度下降）作为优化器，学习率设为 **0.01**，动量设为 **0.9**。



```
============================================================
4. 神经网络基础
============================================================

--- 4.1 定义神经网络 ---
网络结构:
NeuralNetwork(
  (flatten): Flatten(start_dim=1, end_dim=-1)
  (linear_relu_stack): Sequential(
    (0): Linear(in_features=784, out_features=512, bias=True)
    (1): ReLU()
    (2): Linear(in_features=512, out_features=512, bias=True)
    (3): ReLU()
    (4): Linear(in_features=512, out_features=10, bias=True)
  )
)

总参数数量: 669706
可训练参数数量: 669706

--- 4.2 前向传播测试 ---
输入形状: torch.Size([1, 1, 28, 28])
输出形状: torch.Size([1, 10])
输出(未归一化logits): tensor([ 0.0998, -0.1020,  0.0491, -0.1532,  0.0128,  0.0162, -0.0607, -0.0575,
        -0.0422,  0.0609], device='cuda:0', grad_fn=<SqueezeBackward0>)
预测类别: 0

--- 4.3 损失函数和优化器 ---
损失函数: CrossEntropyLoss()
优化器: SGD (
Parameter Group 0
    dampening: 0
    differentiable: False
    foreach: None
    fused: None
    lr: 0.01
    maximize: False
    momentum: 0.9
    nesterov: False
    weight_decay: 0
)
```

[截图 5: 神经网络结构定义与参数统计]

2.6 训练分类器（FashionMNIST）

训练流程包括：前向传播计算预测值、计算损失、反向传播计算梯度、更新模型参数。每个 **epoch** 结束后，在测试集上评估模型性能。

训练参数：

- 训练轮数**(epochs)**：**5**
- 批次大小**(batch_size)**：**64**
- 学习率**(learning rate)**：**0.01**
- 优化器：**SGD with momentum=0.9**

- 损失函数：**CrossEntropyLoss**

训练过程输出示例：

```
============================================================
5. 训练分类器（FashionMNIST）
============================================================

训练参数: epochs=5, batch_size=64, lr=0.01
开始训练...

Epoch [1/5] Train Loss: 0.5493 | Train Acc: 80.12% | Test Loss: 0.4254 | Test Acc: 84.27%
Epoch [2/5] Train Loss: 0.3802 | Train Acc: 86.03% | Test Loss: 0.4101 | Test Acc: 85.07%
Epoch [3/5] Train Loss: 0.3406 | Train Acc: 87.50% | Test Loss: 0.3708 | Test Acc: 86.76%
Epoch [4/5] Train Loss: 0.3156 | Train Acc: 88.38% | Test Loss: 0.3707 | Test Acc: 86.88%
Epoch [5/5] Train Loss: 0.2934 | Train Acc: 89.08% | Test Loss: 0.3424 | Test Acc: 87.83%

训练完成!
模型已保存到 fashion_mnist_model.pth
```

[截图 6：模型训练过程输出]

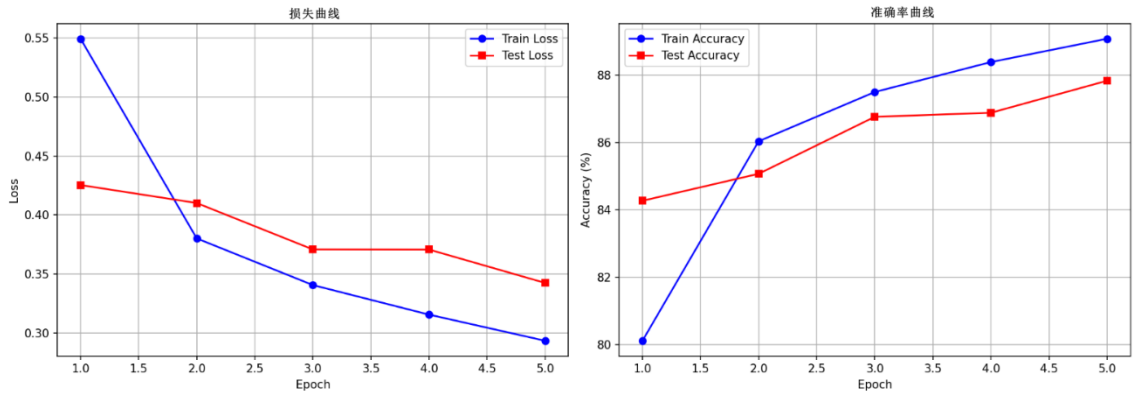
2.7 训练曲线可视化

使用 **matplotlib** 绘制训练过程中的损失曲线和准确率曲线，帮助分析模型的训练状态。理想情况下，训练损失应逐渐下降，训练准确率应逐渐上升。

可视化内容包括：

左图：训练损失与测试损失随 **epoch** 变化的曲线

右图：训练准确率与测试准确率随 **epoch** 变化的曲线

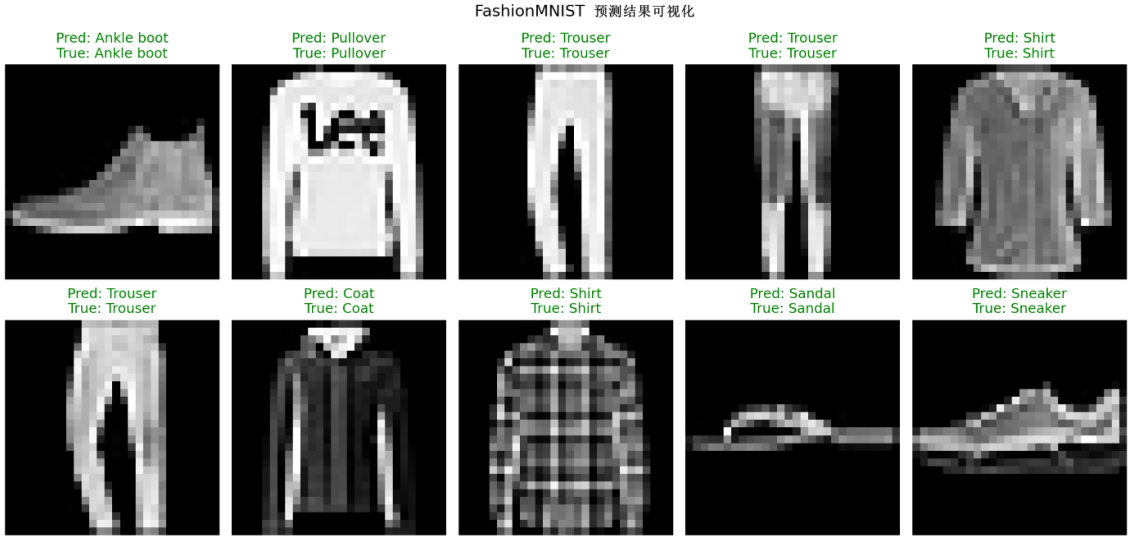


[截图 7：训练与测试损失/准确率曲线图]

2.8 预测结果可视化

从测试集中随机选取样本，展示模型的预测结果。预测正确的样本标题

显示为绿色，预测错误的样本标题显示为红色。通过可视化可以直观地了解模型在各类别上的表现。



[截图 8：FashionMNIST 预测结果可视化图]

2.9 最终评估与各类别准确率

训练完成后，在完整测试集上评估模型性能。

```
--- 5.3 最终评估 ---
最终测试损失: 0.3424
最终测试准确率: 87.83%

各类别准确率:
  T-shirt/top     : 87.70%
  Trouser         : 97.20%
  Pullover        : 84.80%
  Dress           : 83.80%
  Coat            : 80.80%
  Sandal          : 94.80%
  Shirt           : 63.80%
  Sneaker         : 92.40%
  Bag             : 96.90%
  Ankle boot      : 96.10%

============================================================
实验1完成!
============================================================
```

[截图 9：最终评估与各类别准确率]

三、实验结果

本实验成功完成了 **PyTorch** 基础内容的学习与实践。以下是各步骤的关

键结果汇总：

1．环境配置：PyTorch 版本 2.8.0，CUDA 是（是/否）

2．Tensor 操作：成功创建和运算多种 Tensor，验证了 GPU 加速功能

3．DataLoader：成功加载 FashionMNIST 数据集，训练集 60000 张，测试集 10000 张

4．Autograd：正确计算了多项式函数和三角函数的梯度

5．神经网络：构建了含 2 个隐藏层的 MLP，总参数约 669,706 个

6．模型训练：经过 5 个 epoch 训练，最终测试准确率达到 87.83%

7．可视化：生成了训练曲线图和预测结果图

实验总结：

通过本次实验，深入理解了 PyTorch 框架的核心概念和使用方法。Tensor 作为 PyTorch 的基础数据结构,支持在 CPU 和 GPU 之间灵活切换，为深度学习计算提供了高效的张量运算能力。

DataLoader 机制简化了数据批处理和加载流程，使得大规模数据集的训练变得高效且易于管理。Autograd 自动微分引擎是 PyTorch 的核心特性之一，它通过动态计算图实现了灵活的自动求导，极大简化了神经网络中梯度计算的过程。

在 FashionMNIST 图像分类任务中，通过构建多层感知机模型，完整体验了从数据加载、模型定义、训练到评估的深度学习工作流程。实验结果表明，即使是简单的全连接网络，也能在 FashionMNIST 数据集上取得较好的分类效果。

参考文献：

[1] PyTorch 官方文档.

https://docs.pytorch.org/docs/stable/index.html

[2] PyTorch 初学者教程.

https://docs.pytorch.org/tutorials/beginner/basics/intro.html

[3] PyTorch 域库教程.

https://docs.pytorch.org/tutorials/domains.html

[4] Fashion-MNIST: a Novel Image Dataset for Benchmarking Machine Learning Algorithms. arXiv:1708.07747

[5] Paszke, A., et al. PyTorch: An Imperative Style, High-Performance Deep Learning Library. NeurIPS 2019.